

Course Overview

15-213: Introduction to Computer Systems
1st Lecture, Sep. 1, 2015

Instructors:

Randal E. Bryant and David R. O'Hallaron

with Ying Liu's contributions

The course that gives CMU its “Zip”!

Textbooks

■ Randal E. Bryant and David R. O'Hallaron,

- *Computer Systems: A Programmer's Perspective*, **Third Edition** (CS:APP3e), Pearson, 2016
- <http://csapp.cs.cmu.edu>
- This book really matters for the course!
 - How to solve labs
 - Practice problems typical of exam problems

■ Brian Kernighan and Dennis Ritchie,

- *The C Programming Language*, Second Edition, Prentice Hall, 1988
- Still the best book about C, from the originators

Textbook Authors

Randy Bryant



Dave O'Hallaron

Course Components

■ Lectures

- Higher level concepts (<http://csapp.cs.cmu.edu>)
- [YouTube Channel](#)

■ Assignments / Labs

- The heart of the course
- Provide in-depth understanding of an aspect of systems
- Programming and measurement
- This is your focus

■ Exams (midterm + final)

- Test your understanding of concepts & mathematical principles

CMU vs BCC

■ CMU

- Conceptual overview
- Motivation
- Big-picture thinking
- Preparing students for a rigorous systems sequence
- Treat CMU slides as the “WHY”

■ BCC

- Performance formulas
- Quantitative reasoning
- Homework-style problem solving
- Treat BCC assignments as the “HOW”

Getting Help

- Ying Liu: yingliu@peralta.edu
 - Please send me emails in **Canvas**.
 - You can use AI, such as ChatGPT, to learn. You must understand what you are doing, not just copy/past the solutions!
- **Office hours:**
 - After class or by appointment (Zoom)

Policies: Labs And Exams

■ Work groups

- You must work **alone** on all assignments

■ Handins

- Assignments due at 11:59pm on the due day
- Electronic handins using **Canvas** (no exceptions!)
- Please type your solutions. No handwritten answers.

■ Exams

- In-class
- Open book, no computer and internet. Cannot use AI tools.
- Paper-based (answers written by hand)

■ Appealing grades

- In **writing** to Ying Liu within 7 days of completion of grading

Assignments

■ Lateness penalties

- 1+week ☐ 10% penalty
- More than three weeks (21 days) ☐ **No grade**

■ Catastrophic events

- Major illness, death in family, ...
- Formulate a plan (with your academic advisor) to get back on track

■ Advice

- Once you start running late, it's really hard to catch up

Policies: Grading

- **Exams (50%): midterm (25%), final (25%)**
- **Assignments (50%)**
- **Extra Credits (3% max)**
 - attendance
 - optional assignments
 - extra project
- **Final grades based on a straight scale.**
 - A: 90–100%, B: 80–89.9%, C: 70–79.9%, D: 60–69.9%, F: Below 60%

Tentative Course Schedule

- We have a lot of contents to cover
- Chapter 1 to 3 before Midterm
- Midterm TBD (Week 9 or 10)
- Chapter 5, 6, and 9 after Midterm
- Final week for the Final exam

Academic Dishonesty and Cheating

- Please attend all lectures and be prompt.
- All assignments and projects must be done individually.
 - No copy solutions from other students.
 - No copy solutions from the Internet.
 - No use public services or archives to obtain solutions.
 - If you are caught, the penalties WILL be severe. You will receive at the minimum an F in the course and a letter on your university record documenting the incidence of cheating.
 - If you are willing to share your work so that someone else can cheat using your work, please remember that BOTH the giver and the receiver of code are equally culpable and suffer equal penalties.

Others

- No cell phone usage in class.
- Computers must be used for class and lab work only. You must take all your work on a flash drive and do not leave files on the hard drive on the lab computers. You can lose the files and also allow others to copy from them.
- Unless instructed to do so, computers must be turned off during lectures.
- You are encouraged to discuss assignment work with other students. However, all assignments must be done independently.

Ch1: A Tour of Computer Systems

■ Topics

- Introduces how programs are compiled, executed, and managed by hardware and the OS, emphasizing abstraction, memory hierarchy, and performance.
 - From Source Code to Running Program
 - Hardware Organization
 - Memory Hierarchy
 - The Operating System (OS)
 - Processes and Context Switching
 - Virtual Memory
 - Files
 - Networking

Ch2: Representing and Manipulating Information

■ Topics

- How numbers are represented as bits, how C interprets those bits, and why misunderstanding signedness, overflow, and conversion leads to real bugs.
 - Information is Bits
 - Bytes and Word Sizes
 - Unsigned Integers
 - Signed Integers (Two's Complement)
 - Signed vs Unsigned Conversion
 - Bitwise Operations
 - Floating Point



Ch3: Machine-Level Representation of Programs

■ Topics

- Explains how high-level programs are translated into machine-level instructions, showing how the CPU executes arithmetic, control flow, procedures, and memory operations through assembly language.
 - Instruction set architecture (ISA)
 - Registers
 - Assembly language
 - Data movement
 - Arithmetic & logic
 - Control flow
 - Procedures (functions)
 - Memory access
 - Machine-level optimizations
 - Understanding compiler output

Ch4: Processor Architecture (optional)

■ Topics

- Explains how CPUs are built to execute instructions efficiently, introducing datapaths, control logic, and pipelining as the foundations of modern processor design.
 - Instruction Set Architecture (ISA) vs Implementation
 - Control Logic
 - Sequential (Single-Cycle / Multi-Cycle) Processors
 - Pipelining
 - Exceptions and Control Flow
 - Performance Tradeoffs



Ch5: Optimizing Program Performance

■ Topics

- Explains how to make programs run faster by understanding how compilers, processors, and memory systems interact, and by applying performance-aware coding techniques..
 - Measuring Performance
 - The Compiler Is Your First Optimizer
 - Understanding the Processor Pipeline
 - Eliminating Inefficiencies in C Code
 - Reducing Memory Accesses
 - Loop Unrolling
 - Minimizing Branches

Ch6: The Memory Hierarchy

■ Topics

- Explains how virtual memory provides each process with a large, uniform, and private address space while allowing the operating system and hardware to manage physical memory efficiently
 - Physical and Virtual Addressing
 - Address Spaces
 - Virtual Memory as a Tool for Caching
 - Page Tables and Page Translation
 - Translation Lookaside Buffers (TLBs)
 - Memory Mapping
 - Dynamic Memory Allocation
 - Garbage Collection
 - Memory-Related Errors in C Programs

Ch9: Virtual Memory

■ Topics

- Explains how memory is organized into a hierarchy of storage technologies and how programs achieve good performance by exploiting locality
 - The Memory Hierarchy
 - Locality
 - Cache Memory
 - Cache Mapping
 - Cache Performance Metrics
 - Writing Cache-Friendly Code

***Welcome
and Enjoy!***